Nome: Vinicius Ramon Ferreira **RA:** 24360978-2

Atividade - Aula 011

Exercício em duplas: o que é impresso? (1/3)

Para cada trecho: **(a)** o que é impresso ou o que acontece? **(b)** qual regra explica?

1 · Java

```java
static int f() {
    try {
        return 1;
    } finally {
        return 2;
    }
}
System.out.println(f());
```

2 · Python

```python
def f(x):
    try:
        r = 10 / x
    except ZeroDivisionError:
        print("A")
        return -1
    else:
        print("B")
        return r
    finally:
        print("C")

print(f(0))
print(f(5))
```

1 - Java



(a) Imprime 2.
(b) O bloco finally sempre é executado, mesmo depois de um return no try. Como o finally também tem um return, ele sobrescreve o return 1 do try, e o método devolve 2. (O return 1 é descartado.)

2 - Python



(a) A saída é:
A
C
-1
B
C
2.0

Na chamada f(0), 10 / 0 lança ZeroDivisionError, então o except imprime "A" e prepara o return -1. Antes de a função realmente retornar, o finally imprime "C". Só depois o print de fora imprime -1. Na chamada f(5), não há erro: r vale 2.0, o else imprime "B", e o finally imprime "C" antes do retorno. Por fim, imprime 2.0.

(b) O else só roda quando o try termina sem exceção. O except só roda se houver exceção do tipo indicado. O finally roda sempre, inclusive quando existe um return no except ou no else, e ele roda antes de a função devolver o valor.

Exercício: o que é impresso? (2/3)

Para cada trecho: **(a)** o que é impresso ou o que acontece? **(b)** qual regra explica?

3 · Go

```go
n, _ := strconv.Atoi("12a")
fmt.Println(n * 2)
```

4 · C++

```cpp
try {
    throw 42;
} catch (double d) {
    std::cout << "double " << d;
} catch (...) {
    std::cout << "outro";
}
```

3 - Go



(a) Imprime 0.
(b) Go não tem exceções, os erros são valores retornados. strconv.Atoi("12a") devolve dois valores: 0 e um erro (invalid syntax). Como o erro foi ignorado com o "_", o programa continua com n = 0, e 0 * 2 = 0. Nada para o programa, então o erro passa em silêncio se o programador não checar.

4 - C++



(a) Imprime "outro".

(b) O throw 42 lança um int. O catch (double d) não captura, porque o catch compara o tipo exato e não faz conversão de int para double. Os handlers são testados na ordem, e como o primeiro não casou, cai no catch (...), que captura qualquer tipo.

Exercício: o que é impresso? (3/3)

Para cada trecho: **(a)** o que é impresso ou o que acontece? **(b)** qual regra explica?

5 · Rust

```rust
let texto = "12a";
let n: i32 = texto.parse().unwrap();
println!("{}", n * 2);
```

6 · JavaScript

```javascript
async function f() {
  throw new Error("falhou");
}
try {
  f();
} catch (e) {
  console.log("capturado");
}
console.log("fim");
```

5 - Rust



(a) Não imprime nada com println. O programa entra em panic e termina com uma mensagem parecida com:
thread 'main' panicked at ...: called `Result::unwrap()` on an `Err` value: ParseIntError { kind: InvalidDigit }
(b) Em Rust o parse() devolve um Result (Ok ou Err), não lança exceção. O unwrap() em um Err causa panic e encerra o programa, então o println nunca é executado. O jeito seguro é tratar com match ou unwrap_or.

6 - JavaScript



(a) Imprime "fim" e depois dá um erro de promessa rejeitada sem tratamento (Uncaught Error: falhou). Não imprime "capturado".
(b) A função f é async, então o throw não é lançado na hora: ele vira uma Promise rejeitada. Como f() foi chamada sem await, o try/catch já terminou antes da rejeição acontecer e não captura nada. O "fim" é impresso primeiro, e depois o erro aparece como unhandled rejection. Para capturar, é preciso usar await f() dentro de uma função async (ou f().catch(...)).